



Autonomous operations on infrastructure you own

A self-hosted **autonomy harness for physical AI**, where humans, robots and agents learn, play and evolve together in simulated and real worlds.

VERSION 2.1 • SEPTEMBER 2026



TRANSCRIPT OF AN ILLUSTRATIVE MISSION

LIVE TRAFFIC WORLD · ECLIPSE SUMO, SIMULATION OF URBAN MOBILITY · LUXEMBOURG SCENARIO

06:41:02 wake timer · hourly review
06:41:03 context slice: 2 active missions · 214 vehicles tracked · signals nominal
06:41:04 episode #4,112 closed · no action required · sleeping

OPERATOR → PILOT

New mission: survey the northern district for congestion buildup between 07:00 and 09:00, and keep it moving. Boundary attached (survey-north.kml); intersection priorities from ops (priorities.xlsx). Interventions stay inside city signal policy. Report with evidence.

06:42:07 wake operator message · 2 attachments
06:42:08 reason this operator has not sent a .kml boundary or an .xlsx priority sheet before; I can structure both
06:42:09 act **duckdb_ingest** priorities.xlsx → ops_priorities · 14 intersections ranked
06:42:10 act **map__validate_geofence** survey-north.kml → WGS84 polygon · 8.2 km²
06:42:11 act **memory_write** missions ← 'survey-north' · objectives, boundary, constraints
06:42:12 reason the report needs camera footage and vehicle counts; four streams cover the district
06:42:13 act **subscribe** sumo://congestion · scoped to the boundary
06:42:14 act **schedule** wake 07:00 · survey window opens
06:42:15 episode #4,113 closed · mission stored in the **world model** · sleeping

07:00:00 wake timer · survey window open
07:00:03 act **recording__create_recording_projection** latest-at → 61 vehicles in boundary · edge N-14 densest
07:00:05 act **stream__run_recording** /world/camera/n14 · task t-2211 · sleeping
07:11:40 wake task result · 1,204 detections · 87 tracks · northbound queue +9%/min
07:11:43 act **duckdb_ingest** detections → demand per approach
07:11:44 act **optimization__solve_milp** · minimize delay · typed signal choices, demand, priorities, and policy constraints
07:11:52 reason the verified solution retimes N-14 and N-9 inside policy and leaves N-2 unchanged because its priority and demand are low
07:11:53 act **sumo__set_signal_phase** N-14 N-9 · run_batch(500) · task t-2212 · sleeping
07:19:26 wake task result · queue at N-14 down **23%** · writing report
07:19:31 report artifact sealed · counts, plan, clip, rationale · shareable under policy

This transcript is illustrative and uses the SUMO reference integration. The sections that follow describe how its wakes, tools, tasks, and artifacts work.

Abstract

Physical AI agents can now plan and act well enough to run real robots. To trust them with that work, an operation needs an autonomy harness, a system that contains what the agents do. Inside it, agents draw on the operation's context and think in simulation before a robot moves. They stay aligned with the people they serve, act only within the limits those people set, and leave a record of every action.

Veoveo is a self-hosted autonomy harness for physical AI, where humans, robots and agents learn, play and evolve together in simulated and real worlds. A gateway outside the model enforces the operator's identity and policy on each tool call. Simulated and real worlds share the same contracts, so a plan an agent works out in simulation runs on the robot through the same tools, and each run feeds a world model that the next run learns from. Veoveo is built on open standards: MCP for agents and tools, OAuth and OpenID Connect for identity, Rerun for recordings, OpenTelemetry for traces, and Kubernetes for deployment. Any compatible model, robot or simulator connects through them without special integration. An operation keeps the whole harness on its own infrastructure, from a single edge host to an air-gapped cluster.

KEYWORDS autonomy harness · physical AI · world model · runtime policy · MCP

Contents

1	Capabilities	5
2	System Model	6
3	The World Model	7
4	The Agent Kernel	9
5	Edge, Access, and the Gateway	11
6	Hosted Servers	12
7	Tasks and Shared Stores	15
8	Recording Hub	16
9	Connecting Robots and Simulators	17
10	Verification and Observability	20
11	Self-Improvement Primitives	20
12	Deployment and Operations	21
A	Component Reference	23
B	Standards and Protocols	24

1. Capabilities

An installation gives an operation agents that work through the tools the operator approves, and it records every step they take. The table lists what an operator can do with it and the section that explains how each capability works.

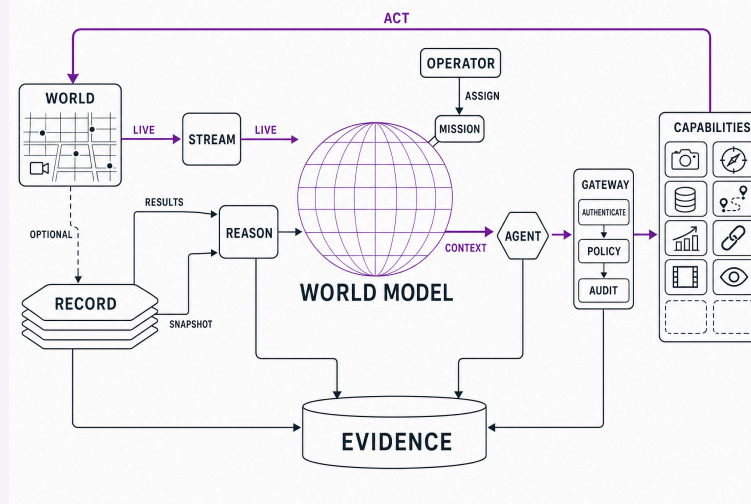


Figure 1. The harness loop. Stream processes live media directly, recording is optional, and every agent action passes the gateway's authentication, policy, and audit checks.

CAPABILITY	WHAT THE OPERATOR GETS	SECTION
Continuous watch	An agent wakes when a watched condition changes, decides, and acts: it retimes a signal, reroutes a vehicle, or reassigns work. Between events it advances the mission on its own schedule.	4
Robots and simulators	Robots, fleets, and simulators connect as MCP servers. Agents command them under the installation's identity, policy, and audit, and each integration can add its own authority, such as a command lease per vehicle.	9
Long work in the background	A render, forecast, route plan, or 500-step simulation batch runs as a task. The agent sleeps until the result arrives, and an interrupted task ends in a state declared before it started.	7
One model of the operation	Sensor data, agent memory, and decisions share queryable stores. An operator can replay any run and see why an agent acted.	3, 8
Rules agents cannot bypass	Policy decides who may call which tool on which tenant's data at which sensitivity. The gateway enforces it outside the model and refuses any call no rule allows.	5
Hosted servers	Media, video, reasoning, SQL, forecasting, optimization, maps, frames, time, speech, computers, simulation, and visualization. A new MCP server joins through a catalog entry.	6
Controlled sharing	Every output is an owned artifact. Owners grant access to users and groups, or publish an expiring, revocable link to a released artifact.	7
Console and Workspace	The Console shows health, tasks, artifacts, agents, recordings, policy, and audit records. Workspace is the chat client where people and agents work together.	12

CAPABILITY	WHAT THE OPERATOR GETS	SECTION
Operator-owned installation	The operator supplies the hostname, identity provider, and object store, and runs the same chart on one host, a cluster, or a disconnected network.	12

2. System Model

The platform runs one loop. Live encoded video enters Stream directly, and a separate forwarder can record it. Agents combine live results, recorded snapshots, and Reason output, then act through tools behind the gateway. The effects appear in the world the agents observe, and each step writes a record. The loop follows Boyd's OODA cycle of observe, orient, decide, and act. Orientation lives in stored state that any agent process can read, which is why a restarted agent resumes with the same picture of the world.

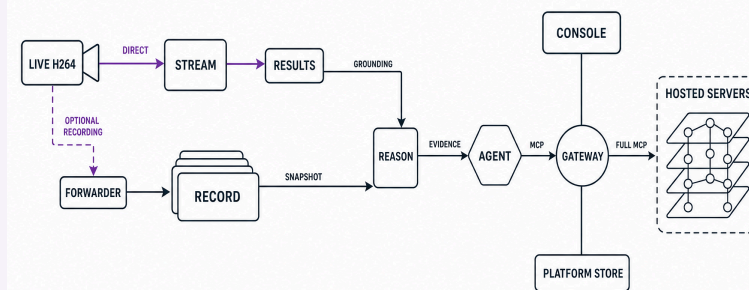


Figure 2. Live H.264 reaches Stream without waiting for the Recording Hub. Stream results and recorded snapshots ground Reason, and agents reach every hosted server through the gateway.

Data moves through the loop in three flows.

FLOW	PATH	GUARANTEE
Observe	Stream processes arriving H.264. A forwarder beside each producer uploads recording batches. Agents read immutable snapshots, taken when a task starts, through Recording and Reason.	Live results never wait for the Recording Hub. The Hub acknowledges a batch after it fsyncs the journal and writes the part.
Act	Every tool call passes through the gateway under one tool namespace. Short tools answer inline, and long tools return a task handle immediately.	A finished task wakes the agent, and the next episode starts with the result.

FLOW

PATH

GUARANTEE

Account Authentication, policy, administration, and task transitions write audit records. Each decision records what the agent saw, what it chose, and why.

Usage records meter provider spend and tool calls per principal.

3. The World Model

An agent's context comes from a world model: what the operation senses, what its agents remember, and what they decided, kept in stores the agent can query. Sensors and simulators stream the physical world into the Recording Hub. Agents keep beliefs, missions, and constraints in their analytical memory, and every decision goes into a decision log. Each action an agent takes becomes data that its next episode reads. The kernel assembles context from these stores at the start of every episode, so an agent keeps what it knows across restarts and beyond the size of any one context window.

The mission is stored as data. Objectives, constraints, and current beliefs sit in tables that the agent reads and writes with guarded tools. When an agent considers a change of plan, it compares the change against the objective the change serves, adopts it only if it still serves that objective, and records its reasoning next to the mission.

A mission can arrive in whatever format its source produces: a scenario file from another autonomy stack, an exported plan, an annotated map, or a plain document. The agent reads the file and writes its objectives and constraints as table rows. Geometry passes through the Map and Frames servers into real coordinates, and the original file is stored as an artifact with its provenance. Connecting an existing autonomy stack starts with giving the agent that stack's files.

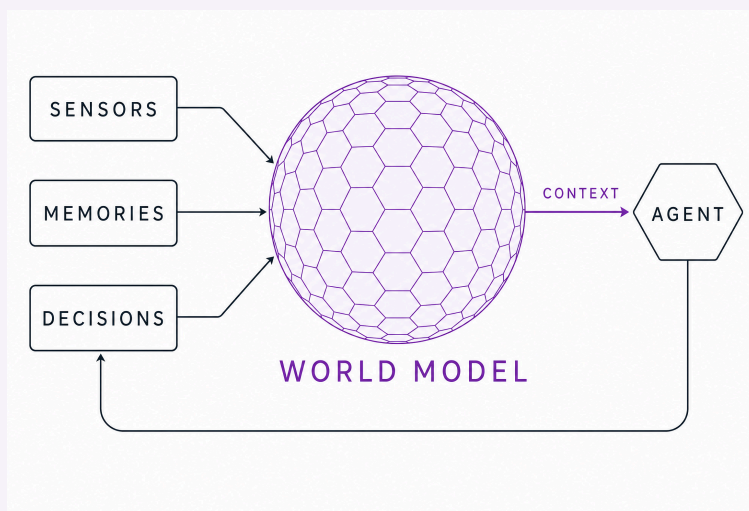


Figure 3. Sensors, memories, and decisions feed the world model. Each episode's context is a query result over it.

SOURCE	WHAT IT CONTRIBUTES	WHAT AGENTS READ
Sensors and simulators	The physical world, streamed into the recorded time-and-space record.	The world now, as a latest-at snapshot, and how it got here, as trajectories and ranges.
Agent memory	Beliefs, missions, constraints, and state in each agent's analytical database.	What the agent believes and intends, through guarded SQL.
Decision log	Every turn, tool call, and rationale, indexed by time and replayable.	What the agents did and why.

3.1 Context, Queried

The kernel builds each episode's context from slices that the agent's manifest defines. The agent holds the same query tools during the episode. When it needs more, it fetches rows from its memory, a snapshot or trajectory from the record, or analytics over data it has ingested.

TOOL	EXAMPLE	WHAT THE EPISODE RECEIVES
memory_query	<code>SELECT id, eta, risk FROM missions WHERE status = 'active'</code>	The active missions as rows. Beliefs and constraints join the same way.
recording__create_recording_projection · latest-at	every vehicle's position, now	A size-limited Arrow snapshot of the world on the recording timeline.
recording__create_recording_projection · range	vehicle 214, last five minutes	One Arrow trajectory on the original timeline.
timeline_query	my tool calls and outcomes, last six hours	The agent's own conduct, as data.
duckdb__query	<code>SELECT class, count(*) FROM detections GROUP BY window_5m</code>	Analytics over any ingested data, including perception output.

Querying for context has two effects. Tokens go to answers, so a long mission history reduces to the rows the current episode needs. Every tool also reads the same stores, which lets results combine: detections land in SQL, SQL rows feed a planning problem, and the committed plan goes back into memory. Section 6.2 follows one such chain.

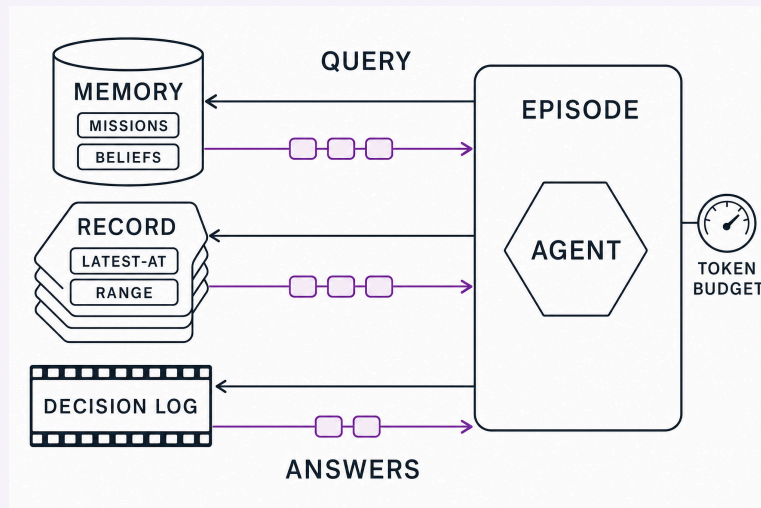


Figure 4. An episode queries memory, the record, and the decision log. Only the answers enter the context, which has a fixed token budget.

4. The Agent Kernel

An agent can run indefinitely because its memory lives in stores that outlast any process. The kernel assembles each episode's context from the world model, and work in flight survives a restart. An agent type consists of a manifest, a preamble, and SQL migrations, and the kernel validates the manifest against its schema before the agent runs. Adding an agent is a configuration change. One installation can run a single agent or a team.

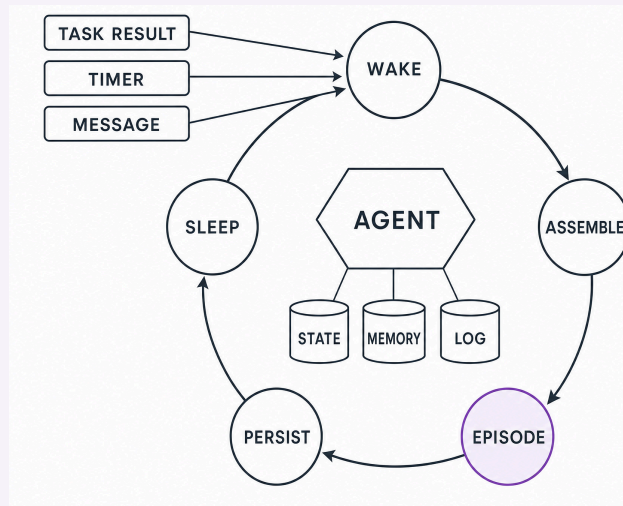


Figure 5. The episode lifecycle: wake, assemble, run, persist, and sleep.

STAGE	WHAT HAPPENS
Wake	The kernel coalesces and deduplicates a batch of wakes. It stores each wake with its disposition, so the ledger shows why each episode ran.
Assemble	The kernel builds the context from current state, beliefs, and history, adds the wake payload, and keeps the total under the token budget.
Episode	The model runs once, within step and token budgets. Short tools answer inline and long tools start tasks. Hooks record each step and stop a run that exceeds its budget.
Persist	Descriptors for unfinished tasks go to the platform store. The episode closes with usage counters and a summary.
Sleep	The agent idles on the wake bus. A leased watcher for each detached task waits for its result, and a heartbeat caps the time between wakes.

Wakes come from task results, resource changes, timers, operator messages, and answers to input requests. Task descriptors and watcher leases reload from the platform store at boot, so a killed and restarted kernel resumes the same tasks.

4.1 Memory and Recovery

Where an agent's memory lives

- **Scheduling state.** The platform store holds episodes, wakes, task watchers, and leases. Its transactional outbox and changefeed deliver wakes between processes.
- **Analytical memory.** Each agent has a sandboxed DuckDB database with its domain tables, a key-value store, and an episode-log view used for context assembly. The Pilot agent's tables are targets, missions, waypoints, constraints, and beliefs.
- **Decision log.** An append-only Rerun recording of every turn, tool call, task event, prompt, and summary. Operators can watch it live.

The agent reads all three itself. `memory_query` runs guarded SQL, `memory_write` applies size-limited mutations and mirrors them to the decision log, and `timeline_query` reads the log as a dataframe.

How work survives interruption

- A task descriptor outlives the episode that created it. A later boot reloads the handle and keeps waiting.
- Token rotation issues the new token before it revokes the old one. Task ids belong to the principal, so a reconnecting agent finds its own tasks.
- Wakes are pushed. In a validation run with polling set to 60 s and the heartbeat to 600 s, an agent using a real model slept through a 15 s task and woke within one second of its completion.
- A run that exceeds its budget ends as `budget_terminated`, which the ledger records like any other outcome. Hourly windows defer wakes that are not marked priority.

AGENTS ARE CONFIGURATION

The manifest declares the model endpoint, the gateway principal, episode limits, budgets, schedule, context sections, and writable tables. The kernel rejects an invalid manifest at boot. Authenticated users and policy-approved service principals reach a running agent through wakes: they can send it a prompt, read its status in the Console, and answer its pending input requests.

5. Edge, Access, and the Gateway

Every client enters through the installation's Kubernetes Ingress. The gateway authenticates each request, checks it against policy, and records the decision before it forwards anything. Policy runs outside the model. A model that is wrong, drifts, or follows injected instructions still cannot call a tool the policy denies. Hosted servers sit on the internal network and accept only requests that carry a gateway-signed Ed25519 identity assertion.

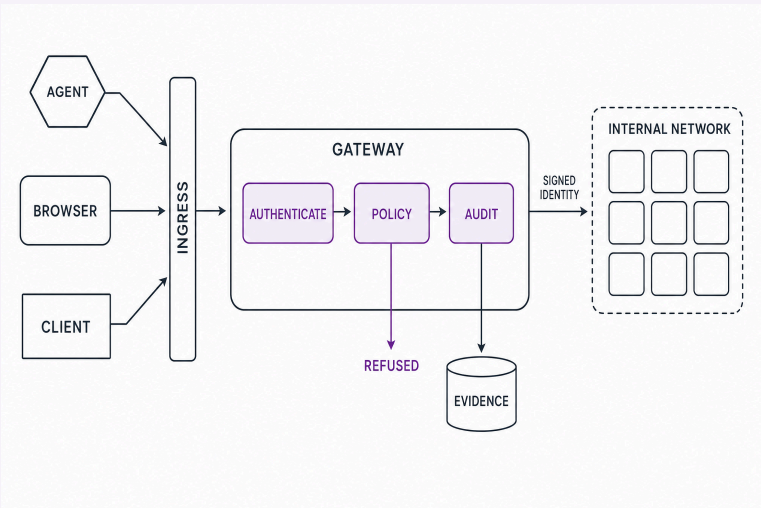


Figure 6. The gateway authenticates each request, checks policy, and writes an audit record. It refuses by default and forwards a signed identity.

PUBLIC PATH	DESTINATION
/mcp · /oauth · /admin · /artifacts	The gateway.
/console · /workspace · /auth	The web backend that serves the Console and Workspace and holds browser sessions.
/s/{token}	The artifact service, for public share links.

FUNCTION	BEHAVIOR
Authentication	Browser OAuth with PKCE, client credentials with private-key JWT, identity-assertion grant (ID-JAG) exchange, JWKS, and token issuance and revocation. Refresh tokens rotate, and reusing an old one revokes its whole family.

FUNCTION	BEHAVIOR
Policy and audit	Attribute- and role-based policy covers tools, resources, tasks, and artifacts, with tenant and data-label checks for regulated data. Every denial carries a reason code, and admin, authentication, and policy operations write audit records.
Control plane	Administrators apply and reload a validated catalog of servers, profiles, and policies while the gateway runs. Secrets are referenced by name and redacted. Artifact downloads pass policy and audit.
Domain administration	Each server administers its domain through MCP tools, resources, tasks, and Apps. A server may also declare an HTTP route at <code>/admin/{profile}/servers/{server}/{path}</code> that uses the same models, policy, audit, and state.
Protocol surface	Tools, resources and templates, prompts, completions, tasks, subscriptions, notifications, structured content with declared schemas, artifacts, and usage. The gateway prefixes tool names when it aggregates servers and leaves resource URI schemes unchanged.

A server that speaks MCP over stdio joins through the Stdio Bridge, which runs it as a child process and serves its tools over streamable HTTP. The Recording Viewer is deployed this way, with the headless viewer, its stdio MCP server, and the bridge in one container.

6. Hosted Servers

The gateway catalog lists every hosted server. Each server declares the MCP surfaces its domain needs and runs behind gateway-signed identity. A local, bridged, or remote MCP endpoint joins by registering in the validated catalog. From then on it uses the same task, artifact, identity, policy, and audit contracts as the built-in servers. A client that connects to the gateway reaches every registered server, and a newly registered server is available to every client.

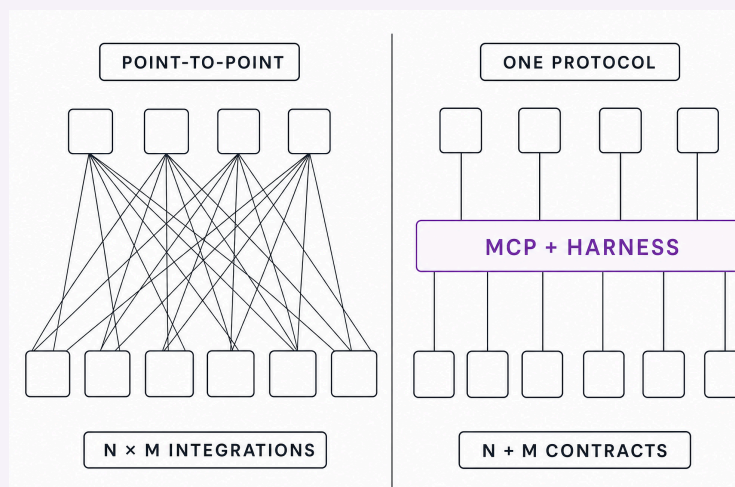


Figure 7. With a gateway, N clients and M systems need N + M connections instead of N × M direct integrations.

SERVER	CAPABILITY	REPRESENTATIVE TOOLS
Media media	Provider-backed generative media with webhook completion, a model catalog, and pricing.	<code>run</code> · <code>models</code> · <code>model_schema</code> · <code>artifact</code>
Stream stream	Live RTP/H.264 processing and reproducible replay of recording snapshots, with an encoded App preview and typed results.	<code>start_live_session</code> · <code>stop_live_session</code> · <code>run_recording</code>
Reason reason	A local world model that answers questions about recording snapshots, optionally grounded by Stream detections.	<code>analyze_recording</code>
Speech speech	GPU speech recognition for private Workspace dictation and recording transcription, with JSON and WebVTT output.	<code>transcribe</code> · <code>start_dictation</code> · <code>finish_dictation</code> · <code>cancel_dictation</code>
Computers computers	Retained computers for people and agents, with lifecycle control, command execution, file transfer, and named agent grants.	<code>create</code> · <code>start</code> · <code>stop</code> · <code>execute</code> · <code>transfer_file</code> · <code>grant_automation</code>
Forecasting timeseries	Forecasts over any SQL-readable time series, returned as Rerun recordings.	<code>forecast</code>
Time time	Civil, military, GNSS, and mission time bound to versioned TZDB and leap-second releases, plus calendars, windows, and clock checks.	<code>resolve_time</code> · <code>convert_time</code> · <code>assess_clock</code> · <code>evaluate_windows</code>
SQL duckdb	Arbitrary SQL over databases the caller owns, inside a locked sandbox with resource limits.	<code>query</code> · <code>execute</code> · <code>ingest</code> · <code>export</code>
Optimization optimization	NVIDIA cuOpt vehicle routing, scenario batches, and convex and MILP solving, with immutable problem, run, solution, and verification records.	<code>optimize_routes</code> · <code>optimize_route_scenarios</code> · <code>solve_convex</code> · <code>solve_milp</code> · <code>verify_solution</code>
Frames frames	ECEF-rooted frame worlds with immutable revisions, coordinate conversion, batch transforms, and provenance.	<code>create_world</code> · <code>publish_world</code> · <code>convert_frame</code> · <code>batch_transform</code>
Map map	Earth geography over versioned data releases and restrictions, with Valhalla and network routing, cuOpt travel models, and reachable areas. Each route records its data, profile, and planner versions.	<code>search_locations</code> · <code>route</code> · <code>route_matrix</code> · <code>build_travel_model</code> · <code>reachable_area</code> · <code>publish_restriction</code>
Datasheet datasheet	The Python reference server: dataset preview, column statistics, and profiling that writes its report as an artifact.	<code>preview_dataset</code> · <code>column_stats</code> · <code>profile_dataset</code>
Sharing artifact	Artifact metadata, access grants, release state, and share links.	<code>metadata</code> · <code>grant_access</code> · <code>revoke_access</code> · <code>set_release_state</code> · <code>create_share_link</code>
Recordings recording	Authorized Rerun datasets, immutable layers, size-limited Arrow projections, subscriptions, and publication.	<code>create_recording_projection</code> · <code>seal_recording</code>
Charts charts	Chart rendering that turns query results into shareable images and chart views.	<code>render_chart</code> · <code>compile_chart</code> · <code>validate_chart</code> · <code>create_chart_view</code>
Viewer rerun	A Rerun viewer driven over MCP: session control, capture, widget inspection, and UI automation.	17 tools, connect through batch
3D View view	Static 3D Tiles scene compositions with fixed inputs and overlays, rendered offscreen on NVIDIA Vulkan.	<code>create_scene_composition</code> · <code>create_view</code> · <code>set_camera</code> · <code>capture_frame</code>
UAV Simulation uav-sim	Reference integration for UAV fleets: simulation sessions, missions, vehicles, terrain, dataset capture, and simulator-hosted cameras streamed as authenticated H.264.	<code>configure_world</code> · <code>execute_vehicle_mission_plan</code> · <code>list_live_cameras</code> · <code>open_live_view</code>

Each server's `DESIGN.md` under `servers/` lists its full tool, resource, and task surface.

6.1 Live Stream, Replay, and Grounded Reason

Stream starts processing RTP/H.264 frames as they arrive. Its encoded tee feeds an App preview and can feed the recording forwarder, and live results never wait for the Recording Hub. The same admitted pass-through or perception profile can run later over a snapshot of complete, acknowledged recording parts, taken when the task starts.

- **Live.** An owner starts one session. Stream processes new frames on the GPU and publishes typed detections, and the H.264 App draws them as overlays.
- **Replay.** The same approved profile runs over an authorized recording range and publishes immutable JSON and RRD results.
- **Reason.** Reason combines an authorized recording snapshot with a typed subset of one `stream://artifact/{id}` result, and its answers can cite Stream track identities.

Stream processing, Reason inference, and visual acceptance fail closed when their GPU is missing. A browser may decode H.264 in software only when Media Capabilities reports the exact configuration as supported and smooth, and the App then labels the path as software decode.

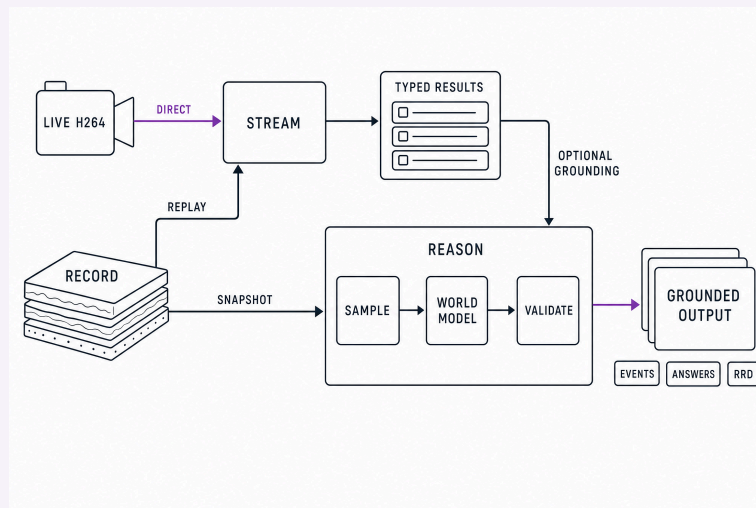


Figure 8. Live Stream and replay publish results. Reason combines a recording snapshot with optional typed Stream detections.

6.2 Optimization

Optimization solves vehicle routing and other mathematical decisions. Routing assigns orders to a mixed fleet and sequences every stop. Convex and mixed-integer models cover decisions that do not fit a route. Typical questions:

- Three survey drones, forty waypoints, and battery limits: which drone flies which route?

- Given measured demand per approach, which signal-timing plan minimizes total delay?
- Ten targets, two sensors, and shifting priorities: what gets watched this hour?

Map publishes immutable cost and transit-time matrices for each vehicle type. An agent may derive coefficients from SQL observations, but it submits the problem as typed JSON with stable identities for every location, order, vehicle, variable, and constraint. The result is an immutable solution linked to the problem and to the run that produced it.

The Rust control plane validates and compiles the typed model, then NVIDIA cuOpt 26.08 solves it on a GPU. Routing supports service and pickup-delivery orders, time windows, breaks, capacities, optional orders, restrictions, warm starts, and weighted objectives. The mathematical solvers accept continuous LP, QP, QCQP, and SOCP models and linear MILP. A separate verifier recomputes route feasibility, bounds, integrality, constraints, and objective values before the solution is stored as a result.

An illustrative signal-retiming run

#	CALL	RESULT
1	wake · resources/updated: sumo://congestion	The congestion resource changed, which wakes the agent.
2	recording__create_recording_projection · latest-at	An Arrow snapshot shows edge E14 saturated.
3	stream__run_recording /world/camera/e14 · 10 min	1,204 detections and 87 tracks of cars, buses, and trucks, counted per approach.
4	duckdb__ingest → query	Demand per approach, as SQL rows.
5	optimization__solve_milp · signal choices and policy constraints	A signal-timing solution, solved on the GPU and independently verified.
6	sumo__set_signal_phase · run_batch(500)	The new phases apply, and the agent sleeps through the batch.
7	wake · task result	Delay at the intersection fell 23%. Every step is already in the record.

7. Tasks and Shared Stores

Tasks

Every long-running tool runs on the shared task runtime through official MCP Tasks. Task ids are UUIDv7. Creation, leases, transitions, results, and outbox events commit atomically in the platform store. A task-augmented `tools/call` returns a handle immediately, `tasks/get` returns status, progress, and the final result, and `tasks/cancel` stops the task and records the provider's cancellation outcome. Each operation declares how it recovers, so an interruption always ends in a known state:

RECOVERY CLASS	MEANING
resume	Deterministic work without side effects may be reclaimed after its lease expires.
webhook_wait	A submitted provider job completes only when its signed webhook arrives.
interrupted_indeterminate	Interrupted work that changes external state fails closed and never replays automatically.

Shared stores

- **Platform store.** One SurrealDB 3.2.4 node on RocksDB coordinates the installation. It holds identity, policy, control-plane revisions, tasks, artifacts, the recording catalog, map catalogs and releases, agents, wakes, audit records, and usage. Cross-process work travels through its transactional outbox and replayable changefeed.
- **Artifact plane.** Every server reads and writes artifact bytes through one policy enforcement point. Bytes live in an S3-compatible object store, and grants, labels, and release state live in the platform store. Each stored occurrence receives a new `artifact://{uuidv7}` identity, and content hashes provide integrity and deduplication within a tenant. One server consumes another's output by its `artifact://` reference, as when DuckDB ingests a media render.
- **Contract core.** Every server shares the same Rust newtypes and enums, deployment models, exported JSON Schemas, and Ed25519 identity assertions. A server refuses to start when validation fails, and it holds only public verification keys.

SHARING

An owner shares an artifact in one of two ways. A grant gives read, write, or admin access to users or groups, and tenant and label policy still apply. For an artifact marked releasable, the owner can also create a read-only link that expires, limits downloads, and can be revoked.

8. Recording Hub

Each producer sends native Rerun traffic to a forwarder on its own loopback interface. The forwarder keeps a size-limited persistent queue, authenticates through the gateway, and resumes uploading from the Hub's checkpoint after a producer or network restart. The Recording Hub fsyncs the batch journal and writes an immutable, ordered RRD part before it acknowledges a batch, and only then may the producer discard its copy. A writing segment closes after one hour or at the 192 MiB precompaction limit, on an IDR frame where video decoding can restart. One object-store pass writes the indexed archive shards and a verified footer. Live Stream processing never waits on the Hub.

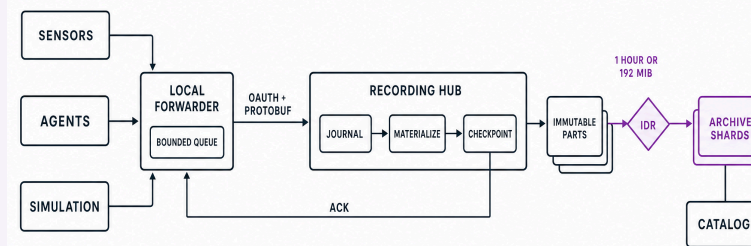


Figure 9. The capture path: fsynced journal, checkpoint, immutable parts, and indexed shards. A 30,307-chunk corpus became 31 archive chunks in 2.56 seconds.

The catalog gives each recording a stable identity in the platform store and links it to its acknowledged parts, archive shards, checkpoints, labels, and lifecycle state. The recording server answers authorized latest-at, range, and dataframe queries, serves subscriptions, and seals a finished recording into an artifact. A replay task reads one immutable snapshot taken when it starts, so later writes cannot change its input.

9. Connecting Robots and Simulators

Robots, fleets, and simulators join an installation as MCP servers, and a physical system connects the same way as a simulation of it. Once connected, a system is available to agents, Stream, Reason, the Console, and the Recording Hub under the installation's identity, policy, and audit. Rehearsals in simulation and fielded runs produce the same kind of record. Two reference integrations, road traffic in SUMO and UAV flight in Isaac Sim, show the pattern end to end.

9.1 The Integration Pattern

An integration lives in the installation's fork, next to the built-in servers.

STEP	WHAT THE INTEGRATOR BUILDS
Control	An MCP server, in Rust or with the Python SDK, that owns the system's control link. Reads and commands become tools, long operations become tasks with a declared recovery class, and watched conditions become subscribable resources.
Authority	Any domain authority the system needs beyond installation policy, such as per-vehicle grants and exclusive command leases.

STEP	WHAT THE INTEGRATOR BUILDS
Record	State and sensor data published to the Recording Hub as Rerun streams through the recording forwarder.
Cameras	Cameras published through the <code>veoveo.io/live-view/v4</code> contract, which gives each viewer its own authorization over one shared encoded product.
Deploy	An image and Helm chart, with a GPU request for any rendering or simulation, a gateway catalog entry, and acceptance tests in the smoke harness.

Registration grants the new server no extra authority. Its tools pass the same identity, policy, task, artifact, and audit checks as the built-in servers from its first request.

9.2 Reference: SUMO Road Traffic

Eclipse SUMO 1.27.1, the Simulation of Urban Mobility, runs the pinned, MIT-licensed LuST Luxembourg scenario in its own container. One `sumo-mcp` server owns the single serialized TraCI connection, publishes the world to the Recording Hub as Rerun `/world/sumo/**` streams, and exposes simulator control as `sumo_*` tools behind the gateway. Its long operations are tasks that the agent detaches from and wakes on, through the same path as every other long tool.

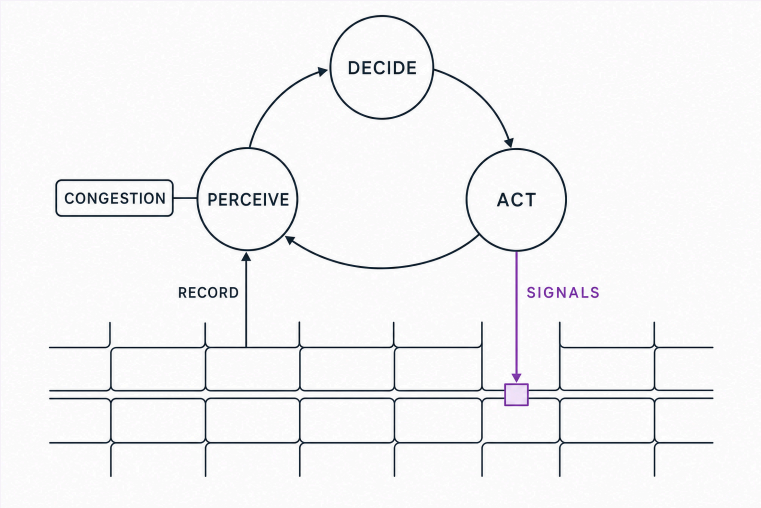


Figure 10. The agent reads the record, decides, and changes signals and routes in the simulated city.

SURFACE	DETAIL
Synchronous tools	<code>query_state</code> · <code>describe_scenario</code> · <code>set_signal_phase</code> · <code>reroute_vehicle</code> · <code>set_edge_speed</code> · <code>close_lane</code> · <code>open_lane</code>
Tasks	<code>run_batch</code> advances the live simulation, so an interruption ends as <code>interrupted_indeterminate</code> . <code>generate_network</code> , <code>compute_routes</code> , and <code>optimize_signals</code> run SUMO command-line programs and resume after interruption.
Events	Agents can subscribe to <code>sumo://congestion</code> , and a forming jam wakes them.

SURFACE

DETAIL

Outputs

A successful task uploads its outputs to the artifact plane with a write capability bound to that task.

9.3 Reference: UAV Flight in Isaac Sim

The UAV reference integration flies a four-vehicle fleet in Isaac Sim 6.0.1 over Google Photorealistic 3D Tiles streamed through Cesium ion. Newton and a batched CUDA Warp plant simulate the airframes, and each vehicle runs PX4 autopilot firmware over MAVLink 2 hardware-in-the-loop. Simulator cameras render on the GPU and encode once with NVENC. The encoded video goes directly to Stream and to authorized viewers, and world state goes to the Recording Hub on a separate path.

The integration adds its own authority on top of installation policy. An agent commands a vehicle only while it holds that vehicle's grant and an exclusive command lease. It resolves destinations through Map, submits a mission plan that the UAV server admits against the route, and executes the plan as a task.

9.4 Measured Behavior

When the agent calls a task-augmented tool, it receives a handle immediately. It stores the descriptor, ends the episode, and sleeps. The server keeps working. When the task finishes, the kernel receives a wake and assembles a new episode with the result.

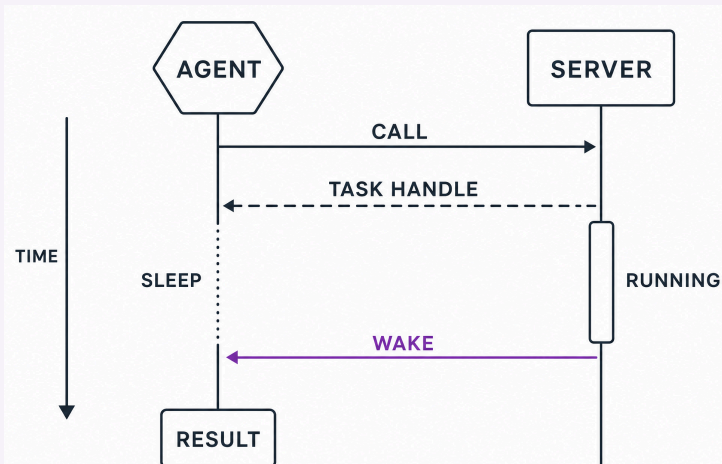


Figure 11. Detach, sleep, and wake. While the task runs, the agent holds no connection and runs no episodes.

PROPERTY

MEASURED BEHAVIOR

**Archive
materialization**

76,094,593 source bytes in 30,307 one-row chunks became 31 archive chunks of 28,815,516 bytes in 2.56 seconds, with 138,812 KiB peak RSS.

PROPERTY	MEASURED BEHAVIOR
Video indexing	194 H.264 GoPs and 3,875 frames became 12 IDR-aligned video chunks of at most 2 MiB each.
Wake latency	With polling set to 60 s and the heartbeat to 600 s, the agent slept through a 15 s task and woke within one second of completion.
Process death	Task descriptors and watcher leases reload at boot. A killed and restarted kernel resumes the same tasks with its memory intact.

10. Verification and Observability

- **Conformance.** A generic MCP client exercises info, resources, prompts, completions, tasks, usage, and artifacts. It also checks authorization-metadata discovery and token handling.
- **Smoke scenarios.** cargo xtask smoke runs end-to-end scenarios with real processes against a fake identity provider, fake providers and upstream services, and a pinned Keycloak realm. The scenarios cover live agent sleep and wake and the full SUMO world in a disposable k3d cluster. The Map scenario acquires and activates fixture data, runs Valhalla and network routes as tasks, and checks that restriction and release changes invalidate dependent resources.
- **Hardware visual acceptance.** Headed browser checks probe WebGPU and WebGL and require at least one hardware-backed path. They reject SwiftShader, llvmpipe, and other software rasterizers. GPU workloads request their accelerator and have no CPU fallback.
- **Telemetry.** OpenTelemetry traces and logs cover the gateway and the servers. Audit summaries give regulated deployments a retained account of admin, authentication, and policy decisions.

11. Self-Improvement Primitives

An operation that runs on the harness produces the data a learning program needs. Every episode leaves a trajectory of the context the agent saw, the action it took, and the result, aligned in time with the world's own record. The harness runs the training program as well. Each installation chooses its method, whether reinforcement learning, imitation, or scenario-driven training, and the objective it optimizes. Every episode is recorded from the day an installation goes live, so a training program starts with the operation's full history.

PRIMITIVE	WHAT THE PLATFORM PROVIDES	WHAT A LEARNING PROGRAM CAN BUILD
Complete episodes	Each episode records its context slice, tool calls, outcomes, and rationale on the same timeline as the sensor record.	State, action, and outcome datasets selected by query, with rewards computed afterward from measured results.

PRIMITIVE	WHAT THE PLATFORM PROVIDES	WHAT A LEARNING PROGRAM CAN BUILD
Perception labels	Stream turns live or replayed video into typed detections and RRD annotations on the source timeline.	Training data drawn from real operations, with provenance for every label.
A practice world	Live and simulated worlds use the same contracts, and recorded anomalies replay as scenarios.	Policy evaluation and rollouts against replayed or simulated conditions, away from the live system.
Measurable objectives	Missions, constraints, and outcomes are queryable, and usage and results are metered per principal.	Objective and reward functions over data the operation already keeps.
Controlled rollout	A candidate agent variant is a manifest, and it acts through the same gateway, budgets, and policy as every other agent.	Shadow or staged variants whose actions stay inside policy, promoted through the control plane.
Lineage	Artifacts carry provenance, and control-plane revisions and the audit trail record what changed, when, and on whose authority.	A traceable chain from data to training run to deployed variant.

12. Deployment and Operations

An installation runs on one host, a cluster, an air-gapped network, or a hybrid of an edge host and remote services. Each form carries the whole stack: the agents, every hosted server, the Console and Workspace, the record, and the audit trail.

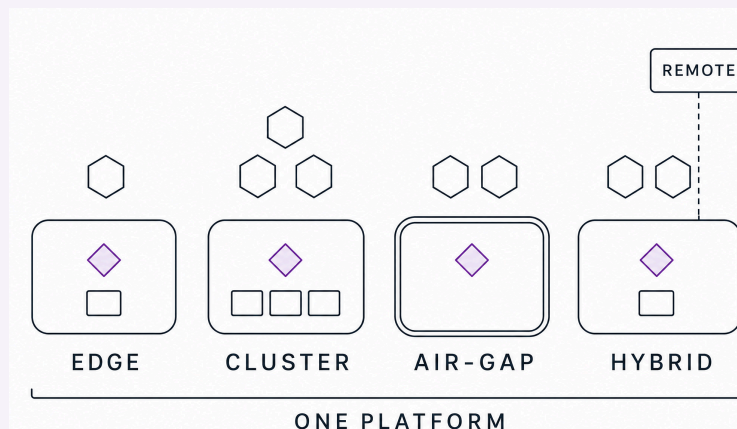


Figure 12. One platform in four forms. Only the hybrid form calls outside the installation, and only to endpoints the operator registers.

FORM SHAPE

Local `cargo xtask smoke profile-cluster-up --profile <profile>` creates a Kubernetes cluster bound to loopback. The same typed profile publishes immutable images and installs the workload model used in the field.

FORM SHAPE

Cluster	The Helm chart runs one SurrealDB StatefulSet with default-deny NetworkPolicy, optional service-mesh mTLS, separate migration and runtime credentials, and telemetry and SIEM wiring supplied by the operator.
Air-gap	The verified offline bundle contains pinned images, Helm material, configuration schemas, checksums, and SBOMs. Installation and verification run entirely inside the disconnected network.
Hybrid	An edge installation registers remote MCP endpoints and provider-backed servers. Calls to them pass the same policy, audit, and task contract as local calls.

Operating an installation

- The Console opens on the running installation: health, tasks, artifacts, agents, recordings, policy, and audit records. Map sources, acquisitions, releases, active pointers, and mobility profiles are managed through Map's MCP tools and its administration App.
- Workspace is the daily chat client. People and agents share conversations, and each person sees their own tool results, task progress, and input requests.
- The web backend handles PKCE login and keeps tokens in an encrypted HttpOnly cookie, so browser JavaScript never receives gateway credentials.
- An operator adds an agent by adding a manifest. A single agent and a team use the same wake bus, budgets, and memory stores in every deployment form.

Appendix A: Component Reference

Role tags: **ours** is built in this repository, **OSS** is an open-source component the platform runs, and **protocol**, **format**, and **concept** name a wire protocol, a data format, and a design idea.

COMPONENT	ROLE	WHAT IT IS
ingress	ours	The installation's single public entry point, a Kubernetes Ingress the operator owns.
gateway	ours	Authenticates callers, checks policy, aggregates every server's MCP surface, and writes the audit trail.
web backend	ours	Serves the Console and Workspace, owns browser login and encrypted sessions, and enforces CSRF protection.
agent kernel	ours	Runs long-lived agents in short episodes and wakes them on a result, a timer, or a message.
world model	concept	What an agent knows: the record, its memory, and the decision log, queried fresh for each episode.
hosted MCP server	ours	A domain service with typed models, full MCP surfaces, and the shared task and artifact contracts.
Stream server	ours	Runs admitted GPU pipelines over live H.264 or one recording snapshot and publishes typed results.
Reason server	ours	Describes recorded segments, detects typed events, and answers questions with a local world model.
Map server	ours	Earth geography and routing over versioned releases, with MCP administration of sources, activation, and rollback.
Python server SDK	ours	Gives Python servers identity checks, official Tasks, artifact capabilities, and usage reporting. Datasheet is its reference server.
platform store	OSS	SurrealDB 3.2.4 holding identity, policy, tasks, artifacts, recordings, agents, audit, and the transactional outbox.
task runtime	ours	Stores long work with atomic leases, transitions, results, and a declared recovery class per operation.
recording forwarder	ours	Accepts Rerun gRPC on the producer's loopback, queues it on disk, uploads through OAuth, and resumes from the checkpoint.
Recording Hub	ours	Acknowledges a batch after fsyncing its journal and writing an immutable part, and advances a monotonic checkpoint.
RRD archive shard	format	An immutable, footer-indexed shard written once at an IDR-aligned one-hour or 192 MiB boundary.
catalog	ours	Links stable recording identities to their parts, shards, checkpoints, labels, and lifecycle state.
recording server	ours	Answers authorized latest-at, range, and dataframe queries and seals recordings into artifacts.
Rerun viewer	OSS	Renders recorded streams as maps, plots, and a scrubbable timeline. The hosted headless instance is the Viewer server.
SUMO	OSS	Eclipse SUMO 1.27.1, a traffic microsimulator running the LuST Luxembourg scenario.
TraCI	protocol	SUMO's control socket for stepping the simulation, reading state, and changing signals and routes.
reference integration	ours	A worked example of connecting a system: SUMO for road traffic and Isaac Sim with PX4 for UAV flight.
sumo-mcp	ours	Owns the simulator's TraCI connection, publishes the world to the Hub, and exposes control as <code>sumo_*</code> tools.
MCP	protocol	Model Context Protocol, carrying tools, resources, prompts, completions, tasks, subscriptions, notifications, and structured content.
task	concept	A tool call the agent can leave: the handle returns immediately, and the result wakes the agent later.
resource subscription	concept	A watched resource whose change wakes the agent, such as <code>sumo://congestion</code> .

Appendix B: Standards and Protocols

The table names each external standard and format the platform exposes or depends on, with the version the implementation pins. Veoveo implements the listed profile of each one, and the owning component's `DESIGN.md` states the exact subset.

STANDARD OR FORMAT	WHERE IT APPLIES
MCP 2026-07-28	The public protocol between clients, the gateway, and hosted servers, including official Tasks. Repository-specific extensions are defined in <code>mcp/contract/DESIGN.md</code> .
OAuth 2.0 and OpenID Connect	Browser login with PKCE, client credentials with private-key JWT, ID-JAG exchange, JWKS, and refresh-token rotation at the gateway.
Ed25519 signatures	Gateway-signed identity assertions that every hosted server verifies with a public key.
JSON Schema 2020-12	Exported contract types and declared tool input and output schemas.
RTP and H.264	Live video into Stream, and camera output from UAV simulation.
Rerun 0.38.1 and RRD	Recording traffic, the decision log, and archive shards.
Apache Arrow	Recording projections returned to agents.
SurrealDB 3.2.4	The platform store, one node on RocksDB.
NVIDIA cuOpt 26.08	Routing and mathematical optimization on a GPU.
Isaac Sim 6.0.1, PX4, and MAVLink 2	The UAV reference integration: simulator, autopilot firmware, and the hardware-in-the-loop link between them.
Eclipse SUMO 1.27.1 and TraCI	The road-traffic reference integration and its control socket.
OpenTelemetry	Traces and logs from the gateway and servers.
Kubernetes and Helm	Every deployment form, including the local k3d cluster and the offline bundle.